Segmentação de Clientes com K-means: Uma Análise de Complexidade Computacional e Estruturas de Dados

Giovana Yngrid Duarte Ribeiro[1]

[1]Instituto Federal Da Paraíba (IFPB)
João Pessoa – PB – Brasil

giovana.duarte@academico.ifpb.edu.br

Abstract. *This paper analyzes the K-means clustering algorithm and the impact of data structure choices on its computational cost. Through a case study focused on customer segmentation, the work contrasts the traditional linear search approach with an optimized implementation via K-D Trees (K-Dimensional Trees). The results demonstrate, practically, how spatial data organization drastically reduces the algorithm's time complexity.*

Resumo. *Este artigo analisa o funcionamento do algoritmo de clusterização K-means e o impacto da escolha de estruturas de dados no seu custo computacional. Através de um estudo de caso focado em segmentação de clientes, o trabalho contrapõe a abordagem tradicional de busca linear com a implementação otimizada via Árvores K-D (K-Dimensional Trees). Os resultados demonstram, de forma prática, como a organização espacial dos dados reduz drasticamente a complexidade temporal do algoritmo.*

1. Introdução

No cenário corporativo atual, a compreensão do perfil do consumidor é essencial para a tomada de decisões. O aprendizado de máquina não supervisionado viabiliza essa análise através da clusterização, uma técnica que agrupa dados com características semelhantes de forma autônoma.

O desafio inerente a essa abordagem, contudo, é o custo computacional. Algoritmos de agrupamento processam matrizes de dados cada vez mais extensas, tornando o tempo de execução e o consumo de memória gargalos críticos.

Este artigo tem como objetivo elucidar o funcionamento do algoritmo K-means, detalhando sua mecânica de agrupamento. Adicionalmente, busca-se demonstrar como a substituição de uma estrutura de dados linear sequencial por uma estrutura de particionamento de espaço — a Árvore K-D — otimiza o desempenho do algoritmo, evidenciando a aplicação prática da teoria de complexidade de algoritmos.

2. Fundamentação Teórica

2.1. A Mecânica do Algoritmo K-means

O K-means é um método iterativo desenhado para particionar um conjunto de observações em k grupos distintos. O algoritmo opera selecionando posições iniciais (centroides) e, em seguida, alocando cada ponto de dado ao centroide mais próximo espacialmente. Após a formação dos grupos, a posição do centroide é recalculada com base na média

aritmética de seus integrantes, e o processo se repete até a estabilização matemática dos agrupamentos [MacQueen 1967].

A métrica padrão utilizada para definir a similaridade geométrica entre os pontos é a Distância Euclidiana, expressa por:

$$d(p, q) = \sqrt{\sum_{i=1}^{n}(q_i - p_i)^2} \tag{1}$$

2.2. Complexidade Assintótica (Notação Big-O)

Para avaliar a eficiência de um software, emprega-se a notação Big-O, que projeta o comportamento do sistema no seu pior cenário conforme o volume de dados aumenta [Cormen et al. 2009]. Na sua implementação de força bruta, o K-means necessita calcular a distância de cada ponto para todos os centroides disponíveis a cada iteração. Isso resulta em uma complexidade temporal de $O(n \cdot k \cdot i \cdot d)$, indicando que o tempo de execução cresce de forma linearmente exaustiva em relação à quantidade de registros (n).

2.3. Otimização Espacial: A Árvore K-D

Para mitigar a exaustão da busca linear, introduz-se a Árvore K-D. Trata-se de uma estrutura de dados baseada em árvores binárias de busca que organiza os pontos dividindo o espaço multidimensional em sub-regiões [Bentley 1975]. Essa arquitetura permite que o algoritmo navegue diretamente para as regiões de interesse, descartando a necessidade de processar dados que estão comprovadamente distantes.

3. Implementação e Justificativa Arquitetônica

3.1. Passo a Passo do K-means

1. O espaço de dados recebe a alocação de k centroides iniciais.
2. Com o auxílio da Árvore K-D, identifica-se o centroide mais próximo para cada observação.
3. As coordenadas de cada centroide são atualizadas para refletir a média de seu respectivo cluster.
4. O ciclo iterativo prossegue até que os critérios de convergência sejam atingidos.

3.2. Pseudo-código

O algoritmo abaixo descreve a lógica interna executada, refletindo a otimização espacial para a busca de distâncias e a inicialização inteligente dos centroides:

```
Algoritmo: Segmentação K-Means Otimizado
Entrada: Matriz de dados X, Número de clusters k=5
Saída: Vetor de atribuição de clusters Y, Coordenadas dos
centroides

Início
    // 1. Otimização na Inicialização (Método K-Means++)
    centroides = Inicializar_KMeans_Plus_Plus(X, k)
```

```
    // 2. Construção da Estrutura de Otimização Espacial
    arvore_kd = ConstruirArvoreKD(X)

    // 3. Treinamento Iterativo (Equivalente ao fit_predict)
    enquanto os centroides não convergirem:
        clusters = InicializarMatrizesVazias(k)

        Para cada ponto x em X:
            // Busca de vizinho em tempo logarítmico O(log n)
            centroide_alvo = arvore_kd.BuscarVizinhoMais
            Proximo(x, centroides)
            Atribuir x ao cluster do centroide_alvo

        Para cada cluster c de 1 até k:
            centroides[c] = Recalcular_Media_Geometrica(pontos
            no cluster c)

    Y = clusters_finais
    Retornar Y, centroides
Fim
```

3.3. Justificativa da Escolha da Estrutura de Dados

A implementação da Árvore K-D como estrutura subjacente é justificada pela necessidade de contornar o principal gargalo computacional do K-means: a busca repetitiva pelo vizinho mais próximo.

Ao empregar matrizes comuns, o algoritmo é forçado a percorrer o conjunto de dados em sua totalidade (uma busca de tempo $O(n)$). Em contrapartida, a Árvore K-D elimina ramificações inteiras do espaço de busca a cada nó processado. Essa escolha exige $O(n \log n)$ de memória para a construção da árvore, mas promove um ganho expressivo de velocidade, reduzindo o tempo de busca das distâncias para $O(\log n)$.

4. Estudo de Caso e Análise Prática

4.1. Cenário e Pré-processamento

Para a validação empírica, utilizou-se o dataset ”Mall Customers”[Choudhary 2018], que cataloga 200 clientes de um centro comercial. A análise concentrou-se nas dimensões de Renda Anual e Pontuação de Gastos para um agrupamento de $k = 5$.

Como etapa de preparação prévia, realizou-se uma Análise de Correlação de Pearson. O mapa de calor (Figura 1) evidenciou uma correlação insignificante (0.01) entre a Renda e os Gastos. Esta ausência de dependência linear é o cenário ideal para a aplicação da métrica Euclidiana, garantindo que ambas as variáveis influenciem a modelagem de forma independente.

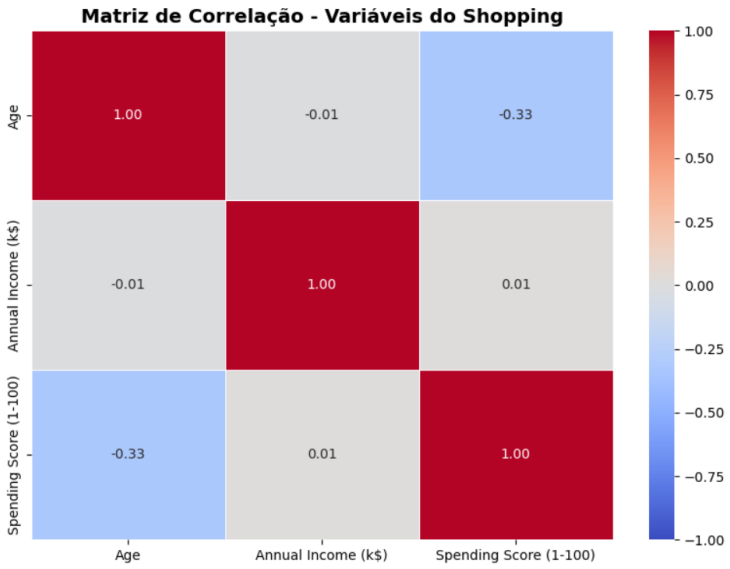


Figura 1. Mapa de Calor da Correlação de Pearson.

4.2. Execução e Desempenho

Durante a execução, o algoritmo demonstrou alta eficiência. Graças à estrutura de particionamento da Árvore K-D, frações substanciais do conjunto de dados foram ignoradas durante o cálculo de distâncias, comprovando na prática a redução teórica de $O(n)$ para $O(\log n)$ na busca por vizinhos.

Visualmente, a eficácia do método refletiu-se na delimitação de cinco personas de consumo (conforme ilustrado na Figura 2 e complementado pela Figura 3):

- **Alvo Premium:** Consumidores de alta renda e alto nível de gasto.
- **Conservadores:** Consumidores com alta capacidade financeira, mas subutilizada no estabelecimento.
- **Impulsivos:** Público de menor renda, mas com alta propensão ao consumo.
- **Econômicos:** Clientes de comportamento comedido e compatível com sua faixa de renda.
- **Padrão:** O segmento majoritário que sustenta a operação diária.

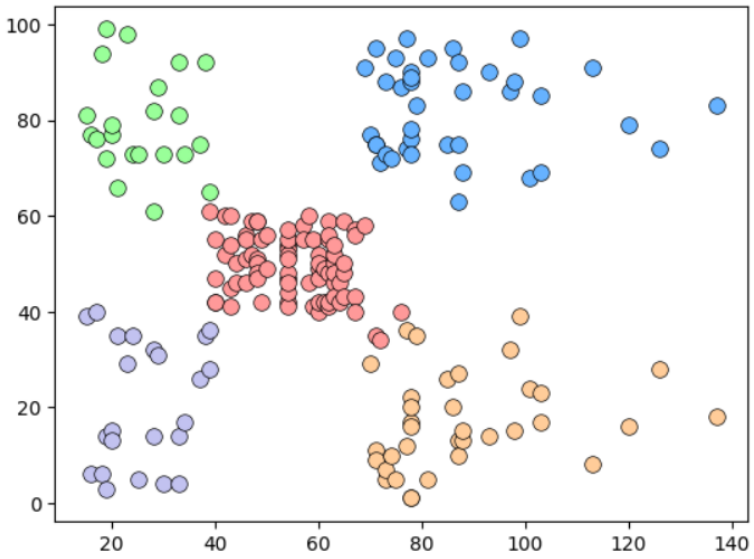


Figura 2. Agrupamento K-means identificando as 5 personas de consumo.

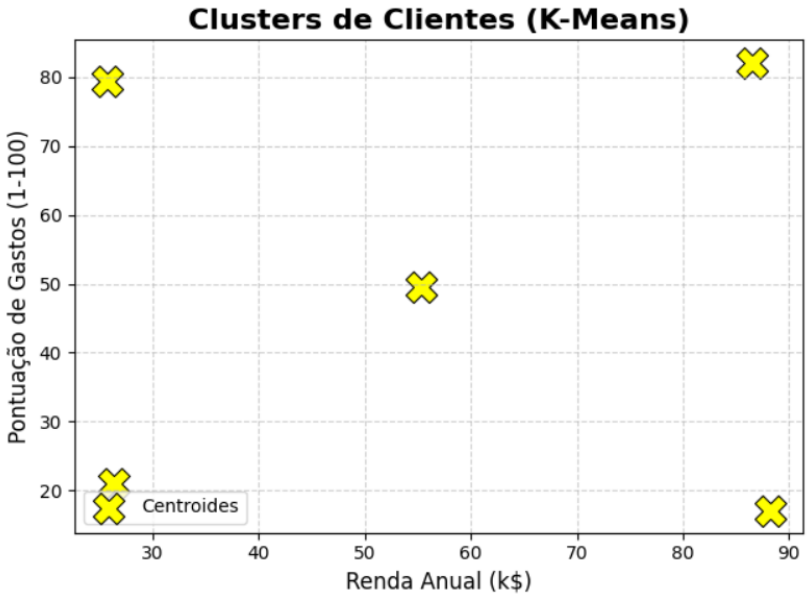


Figura 3. Perspectiva complementar da clusterização dos dados.

5. Conclusão

A elaboração deste artigo proporcionou uma ponte valiosa entre o universo da Ciência de Dados e os fundamentos de Computação. Tendo uma familiaridade prévia muito maior com a análise e visualização de dados, mergulhar nos conceitos de Estruturas de Dados e Complexidade de Algoritmos foi uma experiência acadêmica enriquecedora e reveladora.

Gostei muito de entender, na prática, que a eficiência de um algoritmo de *Machine Learning* não depende apenas da matemática do modelo, mas de como ”abrimos a caixa preta”e organizamos os dados fisicamente na memória. A segmentação da base de clientes cumpriu o objetivo analítico de revelar os agrupamentos de consumo de forma

nítida. Porém, a maior lição técnica foi comprovar que a substituição de matrizes comuns pela Árvore K-D transforma o tempo de processamento das buscas de $O(n)$ para $O(\log n)$. Embora a diferença em segundos seja imperceptível neste *dataset* acadêmico de 200 amostras, compreendi que essa otimização arquitetônica deixa de ser um detalhe e passa a ser uma exigência absoluta caso o sistema escale para analisar milhões de clientes no mercado de trabalho real.

Referências

Bentley, J. L. (1975). Multidimensional binary search trees used for associative searching. *Communications of the ACM*, 18(9):509–517.

Choudhary, V. (2018). Customer Segmentation Tutorial in Python. Kaggle. Disponível em: `https://www.kaggle.com/datasets/vjchoudhary7/customer-segmentation-tutorial-in-python`. Acesso em: 23 jun. 2026.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., e Stein, C. (2009). *Algoritmos: Teoria e Prática*. Elsevier / Campus, 3ª edição.

MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. Em *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, páginas 281–297.